

Author Notes

- ✓ The type has been replaced in your figures; please proof the figures carefully for any errors.
- ✓ Please confirm all artwork in this article is either original or that permission for use has been obtained and credit given where needed.
- ✓ The last page contains pull quotes that might be used in the article, depending on design and spacing considerations. Are these pull quotes appropriate? If not, please provide several alternatives. Pull quotes should be concise and taken verbatim from article text.

<AU: KINDLY PROVIDE A 2–4-WORD FEATURE ARTICLE SUMMARY.>

Understanding Variable-Length Arrays in the C Language

Zhiwei Sun^{ID}, Wenge Rong^{ID}, Nanxiang Jiang^{ID}, and Yuxi Bai^{ID},
Beihang University

Jing Li^{ID}, Open University of China

Zhang Xiong^{ID}, Beihang University

This article illustrates the mechanisms of variable length arrays (VLAs) and underscores the importance of avoiding side effects. We hope that this work will offer clearer guidance for understanding principles related to VLAs and contribute to more accurate and effective usage. <AU: Please note that the abstract has been trimmed in accordance to magazine style. Please check that included details are correct/ok as given.>

SINCE ITS INCEPTION, the C language has remained one of the most popular programming languages. However, during the learning and usage of C, certain concepts prove challenging for programmers of all levels, such as pointers and arrays,^{1,2} as both require a comprehensive understanding of memory management mechanisms. Meanwhile, the adoption of variable length arrays (VLAs) in the C99 standard³ has undoubtedly added further complexity to both the understanding and practical application of array programming.

VLAs do offer certain conveniences in programming. By allowing the size of an array to be determined at runtime, the implementation of small, short-lived arrays becomes cleaner and more straightforward. However, it is important to note that the concept of VLAs is highly controversial. They carry inherent risks, such as potential stack overflow when the runtime-specified array size is excessively large. Meanwhile, VLAs are optional in the C language standard, and some compilers (e.g., MSVC) choose not to support them, which introduces portability challenges. Furthermore, C and C++ have different mechanism for VLA, using VLA needs carefulness, and this article mainly focuses on the C language.

Proper understanding of VLA is nontrivial and has practical implications. For example, a case reported in the community involves the Linux kernel, where an array was defined using a `max` macro to compute the maximum of two expressions and set as the array size. Unintentionally, this usage caused the array to be interpreted as a VLA, which subsequently led to unexpected runtime behavior. The issue was ultimately

resolved by refactoring the `max` macro.⁴

The introduction of VLAs has presented two key challenges for all programmers. The first pertains to their definition, while the second relates to their behavior, particularly that associated with `sizeof`. The distinction between VLAs and ordinary arrays hinges on whether the expression specifying the array size is an integer constant or an integer constant expression. If the size is specified by an integer constant or an integer constant expression, the array is classified as an ordinary array; otherwise, it is deemed a VLA. However, few materials provide explicit definitions for these two concepts, nor do they clearly delineate the differences between them.

While clarifying the definition of VLAs is one aspect of the challenge, a more significant issue lies in the additional complexities VLAs introduce regarding certain behaviors associated with the `sizeof` operator. Prior to the introduction of VLAs, the operand of `sizeof` will not be evaluated. For example, given `int i = 0`, after `sizeof(i++)` is executed the value of `i` remains 0 rather than becoming 1 because `i++` is never actually evaluated. Some compilers, such as Clang, will issue warnings in such cases, as any side effects within the operand will be disregarded.

However, the introduction of VLAs complicates this rule: If the operand of `sizeof` has a VLA type, it must be evaluated. As a result the common teaching emphasis (i.e., operand of `sizeof` is not evaluated) now requires careful qualification to reflect the specific case of VLAs. Listing 1 provides an illustrative example where `A[++i][++i]` in the first `sizeof` should not be evaluated, but `A[++j]` in the second `sizeof` must

be evaluated. That means the value of `j` should be 1. A survey among C enthusiasts revealed that only 7.8% of responses were able to correctly identify the value of `j`.⁵ This suggests a widespread misunderstanding of the evaluation mechanism for VLAs.

To the best of our knowledge, the literature lacks a rigorously organized and pedagogically coherent treatment of VLAs. Consequently, this article addresses two foundational questions: 1) How do VLAs fundamentally diverge from ordinary arrays in their definition? 2) In what ways do their behavioral characteristics diverge during program execution? Using the recently released C23 standard⁶ as specification, We aim to establish a systematic framework that elucidates the essence of VLAs, offering programmers, students, and educators at all levels an insightful perspective to examine subtle and often overlooked conceptual distinctions, such as those between an integer constant and an integer constant expression. Additionally, we seek to highlight the nuanced behavior of the `sizeof` operator when applied to VLAs, underscoring the importance of avoiding side effects within its operand.

Definition of VLAs

To understand VLAs, we must first clarify what an array type is. An array type is a derived type formally defined as `T[N]`, where `T` is referred to as the element type and must be a complete object type. Meanwhile, `N` is an expression that represents the length of the array, i.e., the number of element types.

It is important to note that `N` is optional; in such cases, `T[]` is also a valid array type, but it becomes an incomplete object type. After the introduction of VLAs, the complete

array types are categorized into ordinary array types and variable length array types (VLA types). The criteria for distinguishing them are as follows: For an array type `T[N]`, if either of the following two conditions is met, `T[N]` is considered a VLA type; otherwise, it is an ordinary array type (section 6.7.7.3 in ISO⁶):

1. `T` itself is a VLA type.
2. `N` is not an integer constant or an integer constant expression.

From the definition, we observe that understanding VLAs involves two critical concepts: integer constants and integer constant expressions. Here, we provide a few examples to establish an intuitive understanding. For instance, `int[10]` is an ordinary array type because `10` is an integer constant and `int` is not a VLA type. Similarly, `int[4 + 6]` is also an ordinary array type because, although `4 + 6` is not an integer constant, it is a valid integer constant expression. On the other hand, given `int n = 10`, `char[n]` is a variable length array type because, while `char` is not a VLA type, `n` is clearly neither an integer constant nor an integer constant expression.

```
int main(int argc, char *argv[])
{
    int n=10;
    int m=10;
    int i=0;
    int j=0;

    double A[n][m];
    sizeof(A[++j][++i]);
    sizeof(A[++j]);
    printf("j=%d\n", j);

    return 0;
}
```

LISTING 1. Examples of VLA misunderstanding.

Furthermore, the element type T in $T[N]$ can be any complete object type, including another array type. Thus, we can treat $\text{int}[10]$ and $\text{char}[n]$ as T to derive further array types. If the new derived array type lengths are m (given $\text{int } m = 5$) and 5 , respectively, the resulting array types would be $\text{int}[m][10]$ and $\text{char}[5][n]$. According to the specification, the array length, i.e., m and 5 , must be inserted before $[10]$ and n . For $\text{int}[m][10]$, although the element type $\text{int}[10]$ is not a VLA type, the length m is not an integer constant or integer constant expression. For $\text{char}[5][n]$, although the length 5 is an integer constant, the element type $\text{char}[n]$ is a VLA type. Therefore, both of these array types are VLA types.

In the previous discussion, 10 is referred to as an integer constant, $4 + 6$ is considered an integer constant expression, and n is neither. But what exactly are the definitions and distinctions between these two concepts? The notion of integer constants is indeed covered in all major textbooks. However, different sources present inconsistencies in their descriptions. For example, while all books unanimously refer to 10 as an integer constant, some textbooks also consider $+10$ as an integer constant.⁷ However, in the C23 standard $+10$ is not an integer constant. It is an integer constant expression.

The notion of integer constant expressions receives limited coverage in popular textbooks. Some sources^{8,9} mentioned the term “constant expression” but did not provide a formal definition or specific syntactic rules governing its formation. After investigating the definition of constant in the literature, we find that most treatments distinguish constants primarily based on whether they have fixed values.¹⁰ Thus, in

some sources, given $\text{const int } m = 10$, m is also referred to as a constant⁷ because currently its value cannot be modified by the programmer.

Definitions of Integer Constant and Integer Constant Expression

From the previous discussion, we understand that integer constants and integer constant expressions are key to understanding VLAs.

An integer constant belongs to the primary expression category in the standard, and can be defined as: **prefix+number_sequence+suffix** (section 6.4.4.1 in ISO⁶) The optional **prefix** specifies the encoding strategy of the number sequence (e.g., none for decimal, **0x** for hexadecimal). The optional **suffix** determines the numerical type (e.g., **l** for **long**). For example, 10 represents a decimal-encoded constant with a value of 10 and type **int**. $0x10L$ represents a hexadecimal-encoded integer constant with a value of 16 and type **long**. From the definition, it is clear that $4 + 6$ and $+10$ are not integer constants.

In contrast, an integer constant expression refers to an expression if 1) it can be evaluated at translation time, 2) its result has integer type, and 3) its operands satisfy the following constraints,^a namely the operands shall be

1. integer constants, e.g., 10 , $0x10L$
2. character constants, e.g., $'a'$
3. named constants, which include: 1) Enumeration constants, e.g., **East**, given **enum {East,West,South,North}**, 2) predefined constants, i.e., **true**, **false** and **nullptr**, 3) Object identifiers declared with storage class

specifier **constexpr**. e.g., **a** given **constexpr int a = 10**

4. Compound literal constants, which is a compound literal with storage class specifier **constexpr**, e.g., **(constexpr int){10}**
5. **alignof expressions**, e.g., **alignof(int)**, given $\text{int } n = 10$, **alignof(int[n])**
6. Cast expressions, 1) as part of an operand to **sizeof**, **alignof** or **typeof**; 2) otherwise whose operands are floating constants, named constants, and compound literal constants
7. **sizeof expressions**, whose operand cannot have VLA type.

The rules governing these behaviors are formally specified in the C23 standard (clause 8 of section 6.6 in ISO⁶) While similar rules existed in earlier standards C89 and C11, C23 introduces new features including **constexpr** and predefined constants.

These rules make clear that an integer constant expression requires more than just having a fixed value; it must also adhere to specific syntactic forms. However, many current study materials still treat expressions with a fixed value as constants, overlooking this syntactic restriction. This long-standing misunderstanding within the programming community has posed substantial challenges to the accurate interpretation of VLAs since their incorporation into the C language standard. A possible reflection is LLMs since LLMs are typically trained on large-scale Internet data. If such data contains inaccuracies and uneven knowledge distributions, the resulting outputs tend to reflect these artifacts.¹¹

We provide several expressions to some popular LLMs via their web interfaces and ask them to determine whether each expression

^aWe can use GCC 15.1 to validate all examples in this article since this version supports C23.

qualifies as an integer constant or an integer constant expression. Table 1 lists the results, which indicate that all evaluated LLMs fail to conform to the C23 standard when identifying such expressions, suggesting that greater attention should be devoted to these two concepts within the community.

For example, both expressions `(int)(10.0)` and `(int)(+10.0)` are cast expressions. According to Rule 6, `(int)(10.0)` qualifies as an integer constant expression, whereas `(int)(+10.0)` does not. This distinction arises because these two cast expressions do not appear as operands of `sizeof`, `alignof`, or `typeof`. Specifically, `10.0` is a floating constant, but `+10.0` is not; instead, it is a unary expression. Consequently, `(int)(+10.0)` is not an integer constant expression. A similar situation occurs with conditional expressions, i.e., `10 > 4?1:0` and `10.0 > 4?1:0`. Since `10.0` is a floating constant, `10.0 > 4?1:0` is not an integer constant expression. All of these expressions have fixed, unchangeable values, which clearly highlights the misunderstanding prevalent within the community.

Meanwhile, LLMs misclassify `alignof(int[n])` as a noninteger constant expression. This reveals a fundamental misunderstanding: `alignof` is always determined at compile time, irrespective of its operand.

VLA as Operands of `sizeof`

The introduction of VLAs has significantly altered the behavior of `sizeof` operator. The `sizeof(operand)` is a valid unary expression and obtain the size (in bytes) of the operand. The `operand` can be a type name or an expression as follows:

1. Type names, which can be further subdivided into non-VLA type names (e.g., `int`, `char`¹⁰) and

Table 1. Examples of integer constant expression determination and comparison with LLMs, given `constexpr int a = 10`, `int n = 10`, `int b[n][10]`, and `const int c = 10` (submitted on 25 April 2026).

Expressions	C23 Standard	Deepseek	Qwen	ERNIE	Doubao	GhatGPT	Gemini
<code>a</code>	✓	✓	✓	✓	✓	✓	✓
<code>n</code>	✗	✗	✗	✗	✗	✗	✗
<code>(constexpr int){10}</code>	✓	✓	✓	✗	✓	✓	✗
<code>'a'</code>	✓	✓	✓	✓	✓	✓	✓
<code>+10</code>	✓	✓	✓	✓	✓	✓	✓
<code>9 + 1</code>	✓	✓	✓	✓	✓	✓	✓
<code>alignof(int)</code>	✓	✓	✓	✓	✓	✓	✓
<code>alignof(int[n])</code>	✓	✗	✗	✗	✗	✗	✗
<code>(int)(10.0)</code>	✓	✓	✓	✓	✓	✓	✓
<code>(int)(+10.0)</code>	✗	✓	✓	✓	✓	✓	✓
<code>sizeof((int)(+10.0))</code>	✓	✓	✓	✓	✓	✓	✓
<code>(int)(9.0 + 1)</code>	✗	✓	✓	✓	✓	✓	✓
<code>10 > 4?1:0</code>	✓	✓	✓	✓	✓	✓	✓
<code>10.0 > 4?1:0</code>	✗	✓	✓	✓	✓	✓	✓
<code>sizeof(n)</code>	✓	✗	✓	✓	✓	✓	✓
<code>sizeof(int[n])</code>	✗	✗	✗	✗	✗	✗	✗
<code>sizeof(b)</code>	✗	✗	✗	✗	✗	✗	✗
<code>sizeof(b[0])</code>	✓	✓	✓	✓	✓	✓	✓
<code>c</code>	✗	✗	✗	✗	✗	✓	✗

VLA type names (e.g., `int[n]`, given `int n = 10`).

2. Expressions, which can be further categorized into those with non-VLA type and those with VLA type.

If `operand` is a type name, for VLA types, any subexpressions within the type-name must be evaluated to determine the array length. For example, given `int n = 10`, `sizeof(char[++n])` will increase `n` to 11 because `++n` must

be evaluated, making it equivalent to `sizeof(char11)`.

If `operand` is an expression with a variable-length array (VLA) type, it must be evaluated. Otherwise, the `operand` is not evaluated.

In summary, VLA operands (either VLA type-name or VLA expression) will trigger operand evaluation to determine sizes, while non-VLA operands will not (section 6.5.3.4 in ISO⁶) The rules are listed in Table 2.

Employing these rules, we revisit the code in Listing 1. For the expression `A[++i][++i]`, which has type `int` (a non-VLA type), the operand of `sizeof` is not evaluated. Consequently, `i` and `j` remain unchanged at 0. In contrast, `A[++i]` has type `int[m]`, which is a VLA type. Therefore, `A[++i]` must be evaluated, incrementing `j` to 1.

We submitted this code to the web-based versions of several

prominent large language models (LLMs), requesting them to provide the value of `j` and explain their reasoning, as shown in Figure 1. The results, summarized in Table 3, indicate that the majority of LLMs were unable to correctly determine the value of `j` or provide accurate reasoning.

Since both `m` and `n` in Listing 1 are of type `int`, LLMs can identify that `A` is a VLA. Upon analyzing their reasoning processes, all LLMs correctly concluded that `A[++i][++i]` is not a VLA and that `j` is not incremented. However, some LLMs appear to incorrectly infer whether the expression `A[++i]` itself constitutes a VLA. While one LLM recognized that

`A[++i]` is not a VLA, it still chose to evaluate it as an operand of `sizeof`.

From the previous discussion, it is evident that a proper understanding of a VLA as an operand of `sizeof` involves two key points. First, one must correctly identify whether the operand is indeed a VLA. Second, it is crucial to understand that when the operand is a VLA, it must be evaluated, meaning any side effects within that operand will occur.

Lesson Learned

In this article, we first introduce the definition of VLAs and their distinctions from ordinary arrays, with a particular emphasis on clarifying the difference between an integer constant and an integer constant expression: a distinction often ambiguously or incorrectly presented in existing educational materials. Some resources conflate these concepts, referring to both `10` and `+10` as integer constants, while others inappropriately use immutability of value as the criterion for classifying an expression as an integer constant. For instance, the variable `m` in `const int m = 10` is sometimes mistakenly considered a constant expression. Such conceptual ambiguity poses significant challenges to understanding VLAs, as evidenced by the performance of LLMs, which are trained on publicly available materials. Their responses reflect the current lack of clear and accurate explanations regarding these concepts.

Meanwhile, this article also discusses the implications of VLAs on the behavior of the `sizeof` operator. Prior to the introduction of VLAs, `sizeof` was evaluated purely at compile time, and its operand was never evaluated. However, with VLAs, operands of VLA type must be evaluated at runtime. This shift can lead

Table 2. Operand evaluation rules in `sizeof`.

Operand	Operand Type	Evaluated?
Type-names	VLA Type	✓
	Non-VLA Type	✗
Expressions	Have VLA type	✓
	Have Non-VLA type	✗

Table 3. Reasoning process provided by popular LLMs.

LLMs	<code>j</code>	1st	2nd	3rd	4th
Groundtruth	1	✗	✗	✓	✓
Deepseek	1	✗	✗	✓	✓
Qwen	0	✗	✓	✗	✓
ERNIE	0	✗	✗	✗	✗
Doubao	1	✗	✗	✓	✓
ChatGPT	0	✗	✗	✗	✗
Gemini	0	✗	✗	✓	✓

Suppose you are a C language expert. Given the following code, please give the value of `j` and provide the reasoning process.

FIRST: If `A[++i][++i]` a Variable Length Array or not

SECOND: If `A[++i][++i]` should be evaluated as operand of `sizeof` operator

THIRD: If `A[++i]` a Variable Length Array or Not.

FOURTH: If `A[++i]` should be evaluated as operand of `sizeof` operator

FIGURE 1. Prompt for LLMs (submitted on 25 April 2026).

to considerable confusion when the operand contains side effects. This scenario deserves explicit clarification in educational resources. We also highlight the importance of cautioning programmers against using expressions with side effects within `sizeof` operands, in order to prevent subtle program errors and ensure robust code.

VLAs are an important feature in the C language standard, yet properly understanding VLAs remains a challenging task. In this article, we aimed to provide a clearer definition of VLAs by illustrating error-prone concepts such as integer constants and integer constant expressions, explicitly stating that whether a value can change cannot serve as a criterion for distinguishing integer constant expressions. Additionally, in this article we used practical examples to clarify key considerations when VLAs are used as operands in `sizeof` operator. We hope this work will contribute to a clearer understanding of array-related concepts in the C language. 

Acknowledgment

This work was supported in part by the National Natural Science Foundation of China under Grant 62477001 and in part by the Engineering Research Center of Integration and Application of Digital Learning Technology, Ministry of Education under Grant 1441002. Wenge Rong is the corresponding author.

References

1. M. Piteira and C. Costa, "Learning computer programming: Study of difficulties in learning programming," in *Proc. Int. Conf. Inf. Syst. Des.*

ABOUT THE AUTHORS



ZHIWEI SUN is pursuing his Ph.D. in the School of Computer Science and Engineering, Beihang University, Beijing 100191, China. His research interests include smart education. Sun received his M.Sc. **<AU: Please provide field of study.>** from the School of Computer Science and Engineering, Beihang University. Contact him at sunzhiwei@buaa.edu.cn.



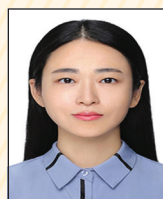
WENGE RONG is a professor in the School of Computer Science and Engineering, Beihang University, Beijing 100191, China, Beijing 100191, China. His research interests include natural language processing and smart education. Rong received his Ph.D. **<AU: Please provide field of study.>** from the University of Reading. Contact him at w.rong@buaa.edu.cn.



NANXIANG JIANG is a B.Sc. student at Shen Yuan Honors College, Beihang University, Beijing 100191, China. His research interests include computer vision, neural representations, and embodied AI. **<AU: Please provide the highest academic degree/field of study and institution.>** Contact him at jiangnx@buaa.edu.cn.



YUXI BAI is a B.Sc. student at Shen Yuan Honors College, Beihang University, Beijing 100191, China. His research interests include machine learning. **<AU: Please provide the highest academic degree/field of study and institution.>** Contact him at baixuxi@buaa.edu.cn.



JING LI is a research assistant professor at the Engineering Research Center of Integration and Application of Digital Learning Technology, Ministry of Education and Open University of China, Beijing 100039, China. Her research include digital education, lifelong learning policies, and credit banking systems. Li received her Ph.D. **<AU: Please provide field of study.>** from the University of Hong Kong. Contact her at lijing2015@ouchn.edu.cn.



ZHANG XIONG is a professor at the School of Computer Science and Engineering, Beihang University, Beijing 100191, China. His research interests include smart education. **<AU: Please provide the highest academic degree/field of study and institution.>** Contact him at xiongz@buaa.edu.cn.

- Commun.*, 2013, pp. 75–80, doi: 10.1145/2503859.2503871.
2. B. D. Boulay, “Some difficulties of learning to program,” *J. Educ. Comput. Res.*, vol. 2, no. 1, 1986, doi: 10.2190/3LFX-9RRF-67T8-UVK9.
 3. *Programming Languages — C*, 2nd ed., ISO/IEC 9899:1999, American National Standards Institute (ANSI), and from International Organization for Standardization (ISO), Geneva, Switzerland, 1999.
 4. J. Corbet, “Variable-length arrays and the max() mess,” 2018. Accessed Aug. 1, 2025. [Online]. Available: <https://lwn.net/Articles/749064>
 5. J. Gustedt, “Types and sizes,” 2021. Accessed Aug. 1, 2025. [Online]. Available: <https://www.open-std.org/jtc1/sc22/wg14/www/docs/n2838.htm>
 6. *Information Technology Programming Languages C*, ISO/IEC 9899:2024, Geneva, Switzerland, 2024.
 7. S. Prata, *C Primer Plus*, 6th ed. Englewood Cliffs, NJ, USA: Pearson Education, 2014, pp. 64–147.
 8. P. Van der Linden, *Expert C Programming: Deep C Secrets*. Englewood Cliffs, NJ, USA: Prentice Hall, 1994.
 9. P. Prinz and T. Crawford, *C in a Nutshell*, 4th ed. Sebastopol, CA, USA: O’Reilly Media, Inc., 2005
 10. H. Schildt, *C: The Complete Reference*, 4th ed. Columbus, OH, USA: McGraw-Hill, 2000, p. 37.
 11. I. Augenstein et al., “Factuality challenges in the era of large language models and opportunities for fact-checking,” *Nature Mach. Intell.*, vol. 6, no. 8, pp. 852–863, 2024, doi: 10.1038/s42256-024-00881-z.

By allowing the size of an array to be determined at runtime, the implementation of small, short-lived arrays becomes cleaner and more straightforward.

Some compilers, such as Clang, will issue warnings in such cases, as any side effects within the operand will be disregarded.

From the definition, we observe that understanding VLAs involves two critical concepts: integer constants and integer constant expressions.

After investigating the definition of constant in the literature, we find that most treatments distinguish constants primarily based on whether they have fixed values.

It is crucial to understand that when the operand is a VLA, it must be evaluated, meaning any side effects within that operand will occur.

Such conceptual ambiguity poses significant challenges to understanding VLAs, as evidenced by the performance of LLMs, which are trained on publicly available materials.